

Mini Jev: An Inspectable Local Interface for Typed Decisions from Frozen Language Models

Yuki Oshio
UPHASH Inc.
oshio@uphash.net

Abstract

Many language-model applications need a small typed decision rather than free-form text. Mini Jev is an open-source local interface that maps a state and explicit criteria to a categorical choice, a true-candidate probability, or an expected ordinal stage. It uses candidate next-token logits from a frozen language model, with no trained decision head. A browser demonstration exposes typed values, candidate distributions, probability semantics, and request/response JSON. Its purpose is to make both the decision and its limitations inspectable. Accompanying artifacts distinguish a 2,400-item self-authored Japanese regression suite from public external evaluation and a 4,050-request matched comparison. Direct readout and native one-token selection agree in all 1,350 pairs; the primary latency interval across 150 local items includes zero. Grammar-constrained JSON is slower in this session, under a different serialization prompt. The contribution is a runnable demonstration and auditable empirical record, not a new learning method or a claim that concentrated candidate probabilities establish correctness.

1 Introduction

A workflow may need a route, an urgency score, or a stage on an explicit rubric. Returning a paragraph and then recovering that value creates an interface problem: the application must validate a result whose useful content is small. A typed interface makes the contract explicit, but a valid type alone says nothing about whether the decision is right. Similarly, a model can assign almost all candidate probability to an incorrect answer.

Mini Jev demonstrates this distinction through a local browser interface and an HTTP/Python API. Users edit a shared state and three questions, run the model, and inspect the resulting values alongside candidate distributions. The inter-

face is inspired by TypeSafe’s Jev (Almeida, 2026; TypeSafe, 2026); this independent implementation makes no claim to reproduce that system’s architecture, training, calibration, compatibility, or performance.

The mechanism is deliberately conventional: a frozen model scores explicit single-token candidates, a restricted softmax produces a distribution, and ordinary code constructs the typed response. Candidate verbalizers and next-token decision scores have substantial precedent (Schick and Schütze, 2021; Zhang et al., 2025). The contribution is the combination of an inspectable local demonstration, explicit probability semantics, and a reproducible comparison that exposes the boundary between interface convenience and evidence about model quality or speed.

The empirical record is also part of the demonstration. A successful schema check, a local regression result, an external-task metric, and a paired runtime measurement answer different questions. We keep them separate. In particular, our matched experiment does not show a material latency advantage over native one-token selection. Source, installation instructions, data documentation, and experiment artifacts are available at github.com/UpHash-Network/mini-jev. Demonstration video: [135-second captioned live recording](#).

2 System and Typed Contract

2.1 From a state to typed values

A request contains a shared state and named questions. Each question supplies a type, natural-language instructions, and candidate criteria. The service returns a semantic label, a candidate distribution, and a type-specific value:

- **Choice** returns the key of the most likely candidate. Keys are canonically sorted before prompting.

- **Noul** exposes the probability of the true candidate in a two-candidate false/true decision.

- **Score** returns the expected zero-based stage of an ordered rubric; the most likely stage is also available as the label.

The distinction between Score’s expectation and its hard label matters. An expectation of 1.6 on stages 0–3 is a probability-weighted position, not an additional model-generated stage or a calibrated utility. Its interpretation assumes equally spaced stage indices; users must decide whether that fits their rubric.

Let z_i denote the next-token logit for candidate $i \in \{0, \dots, K-1\}$, with K candidates and temperature $T > 0$. The service computes

$$p_i = \frac{\exp(z_i/T)}{\sum_{j=0}^{K-1} \exp(z_j/T)},$$

$$s = \sum_{i=0}^{K-1} i p_i.$$
(1)

These are probabilities conditional on the selected candidate set. They are neither probabilities over all valid textual answers nor guarantees that the criteria are exhaustive. Changing the criteria changes the question as well as the normalization. The optional scalar temperature preserves the maximum-logit label, but can change Noul and Score values.

The interface also displays entropy concentration,

$$c = 1 - H(p)/\log K,$$

$$H(p) = - \sum_i p_i \log p_i.$$
(2)

A concentrated distribution has c near one. The display identifies this quantity as concentration rather than estimated accuracy. The response includes probability-semantics and calibration metadata, so downstream applications can retain the same distinction.

2.2 Local execution

The released inference configuration uses Qwen3.6-35B-A3B in Q4_K_M GGUF form (Qwen Team, 2026; ggml-org, 2026), through a source-built llama.cpp helper (ggml-org and contributors, 2026). The model and its output projection remain frozen; there is no trained decision head. The helper performs the full vocabulary projection and then gathers candidate logits. It

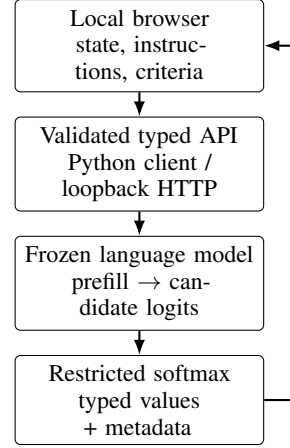


Figure 1: The live demonstration uses the released typed API. It displays model-derived candidate distributions and typed values; no generated explanation or simulated prediction is inserted.

does not implement a selected-row projection optimization.

The prompt repeats the task twice, following the use of repetition for non-reasoning inference (Leviathan et al., 2025). Choice and Noul use letter labels; Score with at most ten stages uses digits after a prefilled answer-object prefix. Larger Score rubrics use letters. The implementation checks that every candidate is a distinct single token at the actual prefix boundary. The API accepts 2–26 candidates; quality evaluation covers 2–8, while larger candidate sets have functional coverage only.

A multi-question request executes its questions sequentially. Model state is cleared between questions: they share the supplied state text, but not a cached prefill. Complete inputs exceeding 2,048 tokens are rejected rather than silently truncated. Schema validation and deterministic response construction prevent free-form output from crossing the API boundary; these software properties do not prevent semantic errors.

2.3 Relationship to one-token decoding

Restricted softmax readout has the same mathematical candidate distribution as a sampler that masks all non-candidates, provided the model prefix, logits, candidate IDs, temperature, and other processors match. Greedy labels then agree unless tie-breaking differs; stochastic sampling agrees in distribution, not necessarily in realized samples. This is an equivalence under specified conditions, not a new inference theorem or learning method.

In particular, selecting a first token after pre-

fill need not require another model forward pass. Avoiding a generated token therefore does not itself establish a speedup. The matched study in Section 4 tests this point using an actual native one-token sampler, rather than comparing direct readout only with a long textual answer.

3 Demonstration and Access

The browser runs locally and connects through a small Python bridge to the existing model API. It requires no cloud inference service. A user starts with a shared state and three editable questions: routing through Choice, urgency through Noul, and priority through Score. The question instructions and criteria are visible, making the decision contract available for inspection before inference.

An example demonstration proceeds in four steps. First, the user reviews the state and defines what the available routes or stages mean. Second, *Run decisions* submits the three questions and displays their typed values. Third, the user examines the candidate probabilities and compares a hard label with the distribution behind it; for Score, the expected stage and the most likely stage can differ. Finally, the user edits the state or a criterion and runs again, inspecting the new response. This is a way to explore behavior, not a controlled accuracy experiment: altering the prompt also alters the model input.

The page displays entropy concentration, measured elapsed time, and copyable request/response JSON. These serve different purposes. The bars explain which candidates receive relative model mass; the JSON exposes the software contract; elapsed time describes the current live request. None is a reference answer or an external quality score. The interface has no replay mode that could be mistaken for live inference. If the model API is unavailable, it reports an unavailable service instead of substituting a canned result.

Installation begins with the repository’s native build script and pinned model downloader. Users build the helper with `build_native.sh`, download the model with `fetch_native_model.py`, and start the API with `./run.sh -without-calibration`. Starting `python3 -m demo.server -port 8766` then opens access at <http://127.0.0.1:8766/>. The repository supplies complete commands and a Python client example. Fresh builds start at $T = 1$; historical calibration artifacts are bound

Evidence	Size	Interpretation
Local regression	2,400 items	Self-authored Japanese; 800 per type
Earlier JNLI	300 items	Balanced public-development pilot
Expanded external	600 items	JCoLA, JSTS, JCQA; 200 each
Matched run-time	4,050 quests	re- 150 local $\times$ 5 repetitions, plus 600 external; 3 modes

Table 1: Distinct evidence sets. The matched study reuses the expanded external items and a subset of the local suite; these counts must not be added as independent evaluations. JCQA abbreviates JCommonsenseQA.

to their recorded runtime and should not silently transfer to a different build.

The tested environment is an Apple M5 Pro with 64 GB unified memory and macOS 26.4, using Metal and Accelerate. The model file is approximately 20.4 GB, in addition to runtime memory. This is a local workstation demonstration, not a claim of lightweight deployment or a demonstrated minimum-memory requirement. Source is distributed; model weights and compiled native binaries are not bundled.

4 Evidence Behind the Demonstration

4.1 Four distinct evidence sets

Table 1 separates the evidence sources. Repeated runtime requests are not additional quality examples, and the external tasks do not share a single meaningful accuracy metric.

The historical local suite contains 2,220 programmatically generated items from 45 template families and 180 individually AI-authored items. Its legacy manual field does not denote human annotation. AI agents reviewed the data, but there was no independent human annotation study. Authoring had access to some earlier development errors and retained related skill families; this suite is useful regression material, not a family-disjoint or fully blind external test.

The historical configuration returned valid typed responses for all 2,400 items and matched the designated top label on 2,238 (93.25%). Type-specific results were 96.00% Choice, 95.25% Noul, and 88.50% Score top-stage accuracy. The 162 errors despite valid outputs illustrate why the demo separates structure from correctness. Positive-temperature scaling fitted on a separate 120-item self-authored calibration split slightly

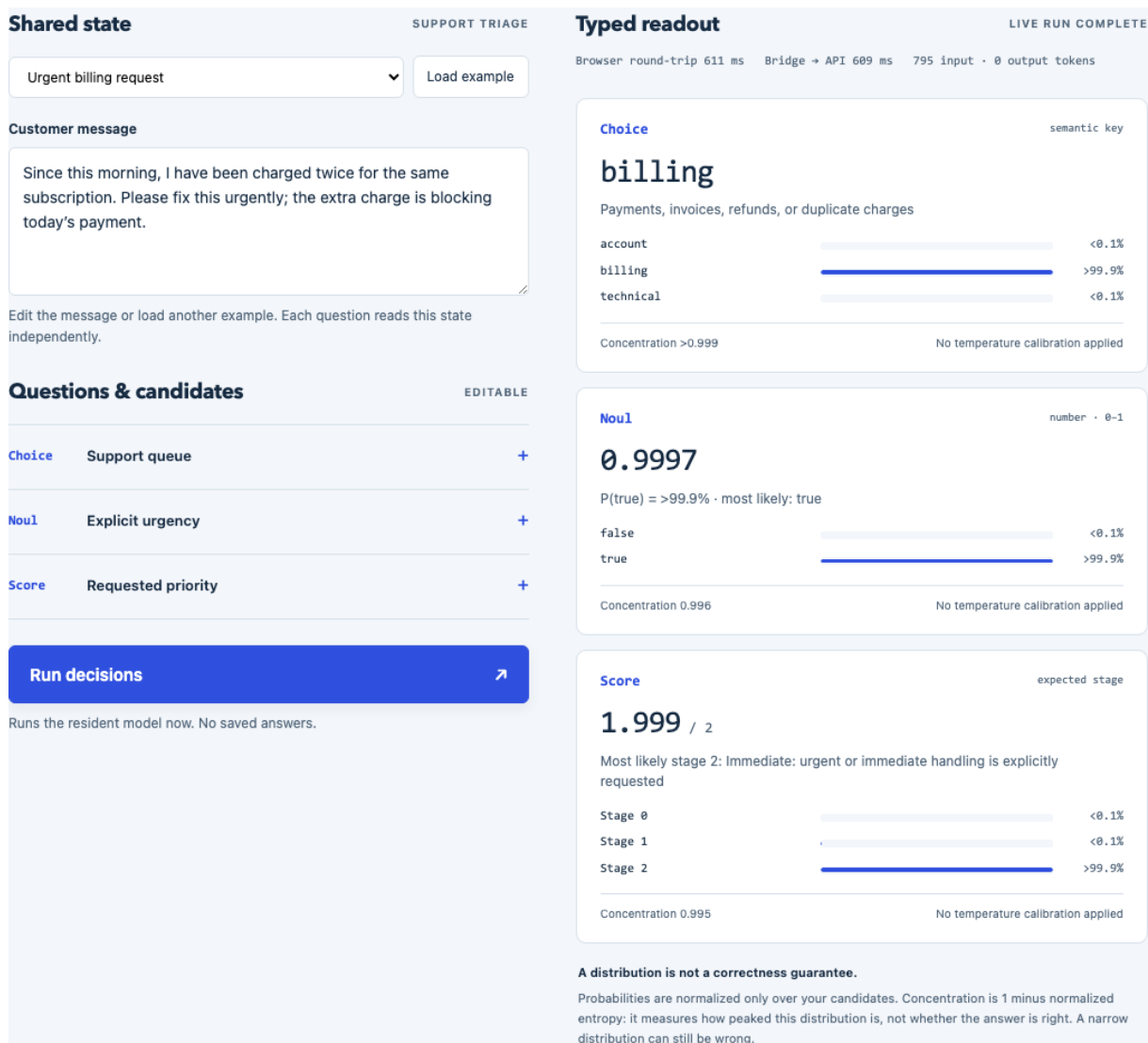


Figure 2: Live browser demonstration: editable inputs, typed outputs, and candidate distributions. The example illustrates the interface and is not an evaluation item.

improved aggregate NLL and Brier point estimates, but worsened ECE and Score expected-stage MAE. Concentration is consequently not presented as a generally validated correctness probability.

The earlier JNLI pilot selected 100 examples per class from public development data (Kurihara et al., 2022), obtaining 243/300 correct (81.0%). Contradiction recall was only 61%. Seventy-seven selected rows share an exact sentence with another selected row, so the pilot is not 300 independent linguistic contexts. It tests Choice on one task, not all three API types.

The expanded sample contains 200 examples each from JCoLA (Someya et al., 2024), JSTS, and JCommonsenseQA (Kurihara et al., 2022). Outcome-independent hash selection and task

rubrics were frozen locally before inference; this was not independent preregistration. Public development data may have appeared in model training. Six JSTS rows share text or image provenance with the earlier JNLI selection, including three identical ordered sentence pairs. We record the overlap rather than claim four independent datasets.

4.2 Matched runtime and parity

All three conditions used one resident model and a common research HTTP response containing a type and semantic label. Direct readout and native one-token greedy selection used identical complete prompts and candidates. The JSON condition actually generated a grammar-constrained object, with a serialization-specific prompt. Its tim-

Comparator – direct	Difference	95% interval
One token	+0.0898	[−0.7483, 0.8449]
JSON	+160.2958	[138.1189, 177.0587]

Table 2: Primary paired HTTP latency differences in milliseconds. Intervals resample observed local family/tag clusters; JSON uses a different serialization prompt.

ing comparison therefore includes prompt and pre-fill differences as well as multi-token generation; it does not isolate sampler overhead.

The schedule contained $(150 \times 5 + 600) \times 3 = 4,050$ measured requests, plus 21 synthetic warm-ups. All measured requests completed with valid responses and no recorded HTTP failures. Mode order was randomized within each item/repetition before execution. Timing ran from client serialization through loopback HTTP response parsing, with a persistent connection and Nagle’s algorithm disabled. Native audit transfer and parsing were inside this interval; the subsequent trace-file write was outside it. These measurements describe the research harness, not the browser’s full rendering time or a production service-level guarantee.

Direct and one-token conditions matched prompts, token sequences, candidate IDs, logits, and labels in all 1,350 paired observations. Maximum observed candidate-logit difference was zero; there were no maximum-logit ties. The primary latency statistic is the median across 150 local items of the within-item difference between five-repetition mean latencies. It is not a difference between marginal medians.

Table 2 gives 10,000-replicate whole-family/tag percentile intervals. They describe sensitivity to the observed cluster composition, conditional on this model, machine, selected items, and session. They do not estimate between-machine or independent-session variability. The one-token interval crosses zero: this run provides no evidence of a material direct-readout latency advantage. JSON was slower under its different serialization requirements, with task-dependent quality changes.

4.3 External task-specific results

Direct readout and one-token selection have identical external labels. JCoLA reached 86.0% accuracy and MCC 0.5708; JSON reached 82.0% and MCC 0.5160. The selected sample contains 158 acceptable sentences, so an always-acceptable

rule already achieves 79.0% accuracy and has undefined MCC. Reporting only accuracy would obscure this imbalance.

JCommonsenseQA accuracy was 95.0% for direct/one-token and 95.5% for JSON, a difference of one example. JSTS retains its continuous reference scores in $[0, 5]$: direct expected-stage MAE at $T = 1$ was 0.5065. JSON has no comparable probability vector, so we do not assign it an expected-stage metric. The separate hard-stage comparison gives MAE 0.5480 for direct/one-token and 0.5650 for JSON. We do not round the gold scores into invented classes or pool these tasks into one accuracy.

The release includes frozen protocols, selections, schedules, source/build hashes, request traces, task-specific analysis, and an independent AI-agent numerical audit. CPU analysis reconstructs published statistics without model execution. Reproduction of inference additionally requires upstream datasets, the pinned model, and a compatible runtime. A fresh build has its own artifact manifest; numerical and timing replication is not guaranteed across hardware or builds.

5 Related Work and Scope

Verbalizer-based classification predates this system (Schick and Schütze, 2021). Qwen3 rerankers provide a concrete precedent for next-token yes/no scoring (Zhang et al., 2025; Qwen Team, 2025). Grammar-constrained decoding also produces structured outputs without task-specific fine-tuning (Geng et al., 2023); our JSON condition is a narrow comparator, not a comprehensive evaluation of constrained decoding.

Scalar temperature scaling (Guo et al., 2017) changes distribution sharpness without changing the argmax. It is distinct from contextual calibration for prompt-induced label bias (Zhao et al., 2021). Multiple-choice models can remain sensitive to option placement (Zheng et al., 2024). Canonical key ordering makes reversed dictionary insertion order equivalent in our software; it does not establish model invariance to semantically re-ordered options.

The demo’s contribution is therefore inspectability and reproducible interface engineering. It neither introduces a new classifier nor establishes superiority over other typed-output systems. The released 35B configuration involves no decision-head training; a separate earlier small-

model head experiment is documented in the repository and is not evidence of training gains for this system.

Limitations

Evaluation uses one quantized model, one Apple Silicon device, and one matched execution session. Quality data are Japanese; an English interface example does not establish English task quality. The public external subsets are limited and have unknown training contamination. The local data were AI-authored and development-related, with no independent human reference validation. There is no user study establishing usability or downstream decision quality.

Candidate probabilities depend on prompts, labels, candidate coverage, and temperature. Concentrated wrong answers occur; neither type validity nor entropy concentration should trigger consequential actions without application-specific validation. Score assumes equally spaced stages and Noul is a relative candidate probability, not a verified frequency of truth. The model is large, questions run sequentially, and other operating systems or lower-memory configurations have not been validated. Live-demo elapsed time is not directly comparable with the research harness’s controlled interval.

Ethics and Availability

The demonstration is intended for inspecting model behavior and prototyping typed interfaces. It provides no evidence for high-stakes deployment. Live inference is local; users still remain responsible for the text they submit and any downstream use. The released code and self-authored local data use MIT terms. External-dataset-derived selection metadata, rubrics, protocols, and results retain the applicable CC BY-SA 4.0 attribution and share-alike terms, separately from generic code. Upstream dataset text, model weights, and compiled binaries are not bundled; the repository’s data cards and notices specify acquisition and attribution.

Acknowledgements

AI coding agents accessed through OpenAI Codex assisted with implementation, question authoring, software checks, experiment execution, analysis, literature organization, and manuscript drafting

under the author’s direction. The independent numerical audit was performed by another AI agent; it is not independent human review. We do not claim human-expert validation of the AI-authored evaluation items. The author is responsible for the submitted manuscript and its claims.

References

- Diogo Almeida. 2026. [Introducing system one models & jev](#). Published 2026-09-15; accessed 2026-09-20.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. [Grammar-constrained decoding for structured NLP tasks without finetuning](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.
- ggml-org. 2026. [Qwen3.6-35b-a3b gguf](#). Pinned Q4_K_M artifact; accessed 2026-09-20.
- ggml-org and contributors. 2026. [llama.cpp](#). Pinned revision used by the native helper.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. [On calibration of modern neural networks](#). In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.
- Kentaro Kurihara, Daisuke Kawahara, and Tomohide Shibata. 2022. [JGLUE: Japanese general language understanding evaluation](#). In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 2957–2966. European Language Resources Association.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2025. [Prompt repetition improves non-reasoning llms](#). *arXiv preprint arXiv:2512.14982*.
- Qwen Team. 2025. [Qwen3-reranker-8b: Official model card and inference example](#). Accessed 2026-09-20; primary source for yes/no candidate-logit extraction.
- Qwen Team. 2026. [Qwen3.6-35b-a3b: Official model card](#). Source-model revision 995ad96eacd98c81ed38be0c5b274b04031597b0; accessed 2026-09-20.
- Timo Schick and Hinrich Schütze. 2021. [Exploiting cloze-questions for few-shot text classification and natural language inference](#). In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 255–269. Association for Computational Linguistics.

- 464 Taiga Someya, Yushi Sugimoto, and Yohei Oseki. 2024.
465 JCoLA: Japanese corpus of linguistic acceptability.
466 In *Proceedings of the 2024 Joint International Con-*
467 *ference on Computational Linguistics, Language Re-*
468 *sources and Evaluation*, pages 9477–9488. ELRA
469 and ICCL.
- 470 TypeSafe. 2026. [Typesafe api reference](#). Accessed
471 2026-09-20; interface inspiration, not implementa-
472 tion equivalence.
- 473 Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin
474 Zhang, Huan Lin, Baosong Yang, Pengjun Xie,
475 An Yang, Dayiheng Liu, Junyang Lin, Fei Huang,
476 and Jingren Zhou. 2025. [Qwen3 embedding: Ad-](#)
477 [vancing text embedding and reranking through foun-](#)
478 [dation models](#). *arXiv preprint arXiv:2506.05176*.
479 Section 2 describes yes/no next-token reranking;
480 inference details are also available in the official
481 Qwen3-Reranker model card.
- 482 Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and
483 Sameer Singh. 2021. [Calibrate before use: Improv-](#)
484 [ing few-shot performance of language models](#). In
485 *Proceedings of the 38th International Conference*
486 *on Machine Learning*, volume 139 of *Proceedings*
487 *of Machine Learning Research*, pages 12697–12706.
488 PMLR.
- 489 Chujie Zheng, Hao Zhou, Fandong Meng, Jie
490 Zhou, and Minlie Huang. 2024. [Large lan-](#)
491 [guage models are not robust multiple choice](#)
492 [selectors](#). In *The Twelfth International Con-*
493 *ference on Learning Representations*. ICLR
494 2024 Spotlight; author manuscript version
495 4, February 22, 2024; conference record
496 <https://openreview.net/forum?id=shr9PXz7T0>.